

Guion — Sustentación Proyecto Integrador

Sistema de Control y Monitoreo de Dron Agrícola

Jeffrey Alejandro Bejarano Parada — Cód. 67001609

Instrucciones de uso:

-  Duración estimada: 12-15 minutos
 -  Abre el repositorio, el dashboard y el Swagger antes de grabar
 -  Alterna entre cámara y pantalla según se indica
 - Las frases entre *[corchetes]* son acciones, no se dicen en voz alta
-

PARTE 1 — PRESENTACIÓN

[Cámara en cara, fondo neutro o escritorio]  ~1 min

"Hola, mi nombre es Jeffrey Alejandro Bejarano Parada, código estudiantil 67001609.

Voy a presentar y sustentar mi Proyecto Integrador de Desarrollo de Software, titulado:

'Dron Agrícola — Sistema de Control y Monitoreo para Dispersión Automatizada de Semillas'.

El objetivo del proyecto es diseñar e implementar un sistema completo que permita controlar un dron terrestre desde una app Android, registrar en tiempo real los datos del sensor inercial MPU6050, almacenarlos en una base de datos en la nube, y analizarlos estadísticamente a través de un dashboard web y reportes en Power BI.

La tecnología principal del backend, como lo requiere el curso, es **FastAPI**."

PARTE 2 — ARQUITECTURA DEL SISTEMA

[Compartir pantalla → README en GitHub, sección "Arquitectura del sistema"]  ~2 min

"El sistema está compuesto por cinco capas que se comunican entre sí:

La **capa 1 es el hardware**: un ESP32 conectado al sensor inercial MPU6050, dos módulos L298N para controlar los motores de tracción y la bomba de agua, y dos servomotores.

La **capa 2 es la app Android**, desarrollada en Kotlin. Envía comandos de movimiento al ESP32 por Bluetooth, y paralelamente hace peticiones HTTP al servidor con los datos del sensor.

La **capa 3 es el backend**, desarrollado íntegramente en FastAPI, corriendo en Render. Aquí se procesan y persisten todos los datos.

La **capa 4 es la base de datos**: SQLite en desarrollo local, y MySQL en Clever Cloud en producción.

Y la **capa 5 es la visualización**: un dashboard web embebido en el HTML, análisis estadístico con pandas y scipy, y reportes en Power BI."

PARTE 3 — REPOSITORIO Y DOCUMENTACIÓN

[Pantalla → github.com/Jeff07K/DRON_AGRICOLA_API] 🕒 ~1.5 min

"Aquí está el repositorio en GitHub. Como pueden ver, el proyecto cuenta con **28 commits**, lo que evidencia trabajo continuo y una iteración progresiva a lo largo del semestre, no fue construido de un momento a otro.

El README está completamente documentado: tabla de contenidos, arquitectura, requisitos de hardware con mapa de pines, instrucciones de instalación y despliegue, referencia completa de la API, comandos Bluetooth, y tecnologías usadas.

La estructura del repositorio incluye: el firmware para Arduino en C++, la app Android en Kotlin, y el backend en Python con FastAPI.

[Mostrar carpetas: *Ardiono*, *android*, y los archivos *main.py*, *models.py*, *database.py*, *operations_db.py*]

Cada componente está separado en su propia capa, lo que refleja una arquitectura limpia."

PARTE 4 — MODELOS Y CRUD

[Pantalla → *abrir main.py* y *models.py* en GitHub] 🕒 ~2.5 min

"Ahora voy a mostrar los modelos y las operaciones CRUD del proyecto.

[Abrir *models.py*]

El modelo principal es **SensorData**, que representa una lectura del sensor inercial MPU6050. Contiene: ID, timestamp, el flag de disparo de la bomba, y los seis ejes del sensor: aceleración en X, Y, Z, y velocidad angular en X, Y, Z.

[Abrir main.py]

Las operaciones CRUD están implementadas así:

— **CREATE**: el endpoint `POST /api/sensor-data` recibe un JSON con los valores del sensor desde la app Android y los persiste en la base de datos.

— **READ**: `GET /api/sensor-data` devuelve todos los registros. `GET /api/sensor-data/{id}` retorna un registro específico por su ID.

— **UPDATE**: en este sistema, los datos del sensor son inmutables por diseño, ya que son registros históricos de mediciones reales. Esto responde a la lógica del negocio: no tiene sentido editar una lectura física pasada.

— **DELETE**: `DELETE /api/sensor-data/{id}` elimina un registro puntual, y `DELETE /api/sensor-data` permite limpiar toda la tabla.

Adicionalmente existe el endpoint `GET /api/export-excel` que genera y descarga un archivo `.xlsx` con todos los datos para análisis externo.

La validación de datos está garantizada por los esquemas de **Pydantic**, que son nativos en FastAPI. Si llega un campo con tipo incorrecto o falta un campo requerido, la API rechaza automáticamente la petición con un error 422 descriptivo."

PARTE 5 — DEMO EN VIVO DEL SISTEMA

[Pantalla → abrir el URL de Render del dashboard] 🕒 ~3 min

"Vamos al sistema desplegado. Esta es la URL pública del proyecto corriendo en Render, accesible desde cualquier dispositivo con internet.

[Mostrar el dashboard web — index.html]

Este es el **dashboard web**, embebido directamente en el HTML que sirve FastAPI. Pueden ver las gráficas con los datos del sensor en tiempo real, las estadísticas de los registros, y el buscador integrado en la interfaz.

Los estilos CSS están aplicados en todo el sitio, dándole una presentación visual coherente.

[Abrir /docs]

Esta es la documentación automática de la API generada por FastAPI con Swagger UI. Todos los endpoints están documentados y son interactivos.

Voy a hacer una demostración en vivo: voy a registrar un nuevo dato sensor.

[Expandir POST /api/sensor-data → clic en 'Try it out']

Ingreso valores coherentes: aceleración X 0.55, Y 1.10, Z 9.74 — estos representan un dron casi nivelado con la gravedad en el eje Z. Disparo en 1, indicando que la bomba estaba activa.

[Clic en Execute]

El servidor responde con **HTTP 201** y el objeto creado con su ID y timestamp. Eso es el CREATE funcionando.

Ahora busco ese registro por ID:

[Usar GET /api/sensor-data/{id}]

Aquí está. Y si quiero verlo en el dashboard, recargo la página...

[Volver al dashboard y mostrar el nuevo dato reflejado]

El dato ya aparece reflejado en la interfaz HTML."

PARTE 6 — DATASET Y ANÁLISIS ESTADÍSTICO

[Pantalla → sección Análisis estadístico del README, luego mostrar el export] 🕒 ~2 min

"Uno de los diferenciadores de este proyecto es el análisis estadístico de los datos reales del sensor.

El endpoint `(GET /api/export-excel)` genera y descarga un archivo `(.xlsx)` con todos los registros almacenados. Ese archivo tiene dos hojas: 'Datos Sensor' con los registros crudos, y 'Estadística' con los indicadores calculados.

[Si tienes el archivo local, mostrarlo. Si no, describir]

Con ese Excel, el script `(analisis.py)` usa pandas y scipy para calcular: estadística descriptiva completa —media, desviación estándar, mínimos, máximos y percentiles— para los seis ejes del MPU6050. También calcula la correlación de Pearson entre Accel X y Accel Y, y hace una regresión lineal entre esas variables, obteniendo la pendiente, el intercepto y el R^2 .

Adicionalmente genera dos imágenes: histogramas de distribución de las seis variables, y un gráfico de dispersión con la línea de regresión.

En Power BI, esos mismos datos se visualizan con gráficas de línea para la evolución temporal, dispersión, barras comparativas y tarjetas KPI de promedios.

Esto enriquece el proyecto al permitir comparar el comportamiento del dron en modo normal versus cuando la bomba está activa, analizando si la vibración del motor de agua afecta la estabilidad del vehículo."

PARTE 7 — VALIDACIONES Y BUENAS PRÁCTICAS

[Pantalla → código en GitHub, main.py y el HTML] 🕒 ~1 min

"En términos de buenas prácticas:

En el **backend**, FastAPI con Pydantic valida automáticamente tipos de datos, campos requeridos y rangos. Los errores son devueltos con mensajes descriptivos en JSON, nunca con stacktraces expuestos al cliente.

La conexión a la base de datos se maneja con sesiones correctamente abiertas y cerradas usando el patrón `yield` de FastAPI. Las variables de entorno sensibles como la URL de la base de datos se manejan con un archivo `.env` y la librería `python-dotenv`.

En el **frontend**, el formulario HTML tiene validaciones de tipo `required`, `type='number'` y rangos donde aplica, para evitar que lleguen datos inconsistentes al servidor.

La data ingresada en el sistema es coherente y representativa: valores de acelerómetro en m/s² y giroscopio en rad/s, tal como los reporta el MPU6050."

PARTE 8 — TECNOLOGÍAS Y CIERRE

[Cámara en cara] 🕒 ~1 min

"Para resumir las tecnologías usadas:

El **firmware** está en C++ sobre Arduino IDE con ESP32. La **app móvil** está en Kotlin con corrutinas para manejo asíncrono. El **backend** está íntegramente en FastAPI con Uvicorn, SQLAlchemy como ORM, y openpyxl para la generación de Excel. La **base de datos** es SQLite en desarrollo y MySQL de Clever Cloud en producción. El **hosting** del backend está en Render. Y el análisis se hace con pandas, scipy, matplotlib y Power BI.

Todo el código, la documentación, los diagramas y los archivos de análisis están en el repositorio de GitHub: github.com/Jeff07K/DRON_AGRICOLA_API.

Este proyecto representa un sistema funcional de IoT agrícola con análisis de datos, que implementa todas las capas requeridas: hardware, móvil, backend, base de datos y visualización.

Quedo atento a las preguntas de sustentación. Muchas gracias."

APÉNDICE — Respuestas a preguntas frecuentes

(Tenerlas presentes, no leerlas en el video)

¿Por qué SQLAlchemy y no SQLAlchemy directamente?

SQLModel fue creado por el mismo autor de FastAPI. Permite definir el modelo de datos una sola vez como clase Python y usarlo tanto como esquema Pydantic (validación de entrada/salida) como tabla de base de datos. Con SQLAlchemy puro necesitarías dos clases separadas.

¿Qué es Uvicorn y por qué se usa con FastAPI?

FastAPI es un framework ASGI, no incluye servidor web por sí solo. Uvicorn es el servidor ASGI que lo ejecuta. Es equivalente a lo que sería Gunicorn para Flask.

¿Cómo maneja FastAPI la validación?

Mediante Pydantic. Cada modelo de entrada hereda de `BaseModel`. Cuando llega una petición, FastAPI deserializa el JSON y lo valida contra el esquema. Si algo falla, retorna automáticamente un 422 con detalles del error sin ejecutar el handler.

¿Por qué MySQL en producción y SQLite en desarrollo?

SQLite es un archivo local, perfecto para desarrollo rápido sin instalar nada. En producción, Render no tiene sistema de archivos persistente entre deploys, así que necesitamos una base de datos externa. Clever Cloud ofrece MySQL gratuito persistente.

¿Qué mide el MPU6050 y cómo se relaciona con el dron agrícola?

El MPU6050 tiene un acelerómetro (mide fuerzas en m/s^2 en tres ejes) y un giroscopio (mide velocidad angular en rad/s en tres ejes). En el contexto del dron, nos permite detectar inclinación, vibraciones y cambios bruscos de movimiento. El análisis estadístico compara el comportamiento cuando la bomba de agua está activa versus inactiva, para evaluar si la vibración del motor afecta la estabilidad.

¿Por qué solo un modelo principal de CRUD?

El sistema es IoT: la entidad central es la lectura del sensor, que es el dato valioso del negocio. El dron en sí es el hardware físico, no requiere persistencia en base de datos. En una siguiente iteración se podría agregar un modelo `Mision` o `Vuelo` para agrupar lecturas por sesión de operación.